TAKYIWAA DONKOR

STUDENT ID: 10798509

OBJECT ORIENTED AND FUNCTIONAL PROGRAMMING WITH PYTHON

HABIT TRACKING APPLICATION

20.03.2026

PROJECT OVERVIEW

Project Overview

- Python-based console habit tracker
- Tracks daily activities and performance
- Designed for simplicity and efficiency

Goal

- Track daily habits
- Analyse performance
- Improve consistency

```
================================
        HABIT TRACKER
================================
1 View Habits
2 Record Habit
3 Longest Streak
4 Current Streak
5 Streak Statistics
6 View Records
7 Daily Report
8 Weekly Report
9 Monthly Report
10 Habit Leaderboard
11 Exit
Select option:
```

CURRENT ARCHITECTURE

Design Features

- Modular structure
- Separation of concerns
- Easy to maintain

Modular System Design

- The project is now structured into modules:

Benefits

- separation of concerns
- easier maintenance
- scalable design

```
+-------------------+
|       USER        |
|   CLI Interface   |
+---------+---------+
          |
          v
+-------------------+
|      main.py      |
|  App Logic & UI   |
+---------+---------+
          |
          v
+-------------------+
|   database.py     |
| DB Operations     |
+---------+---------+
          |
          v
+-------------------+
|     SQLite DB     |
|     habit.db      |
+-------------------+
```

CLASS AND EVENT LOG STRUCTURE

Habit Representation

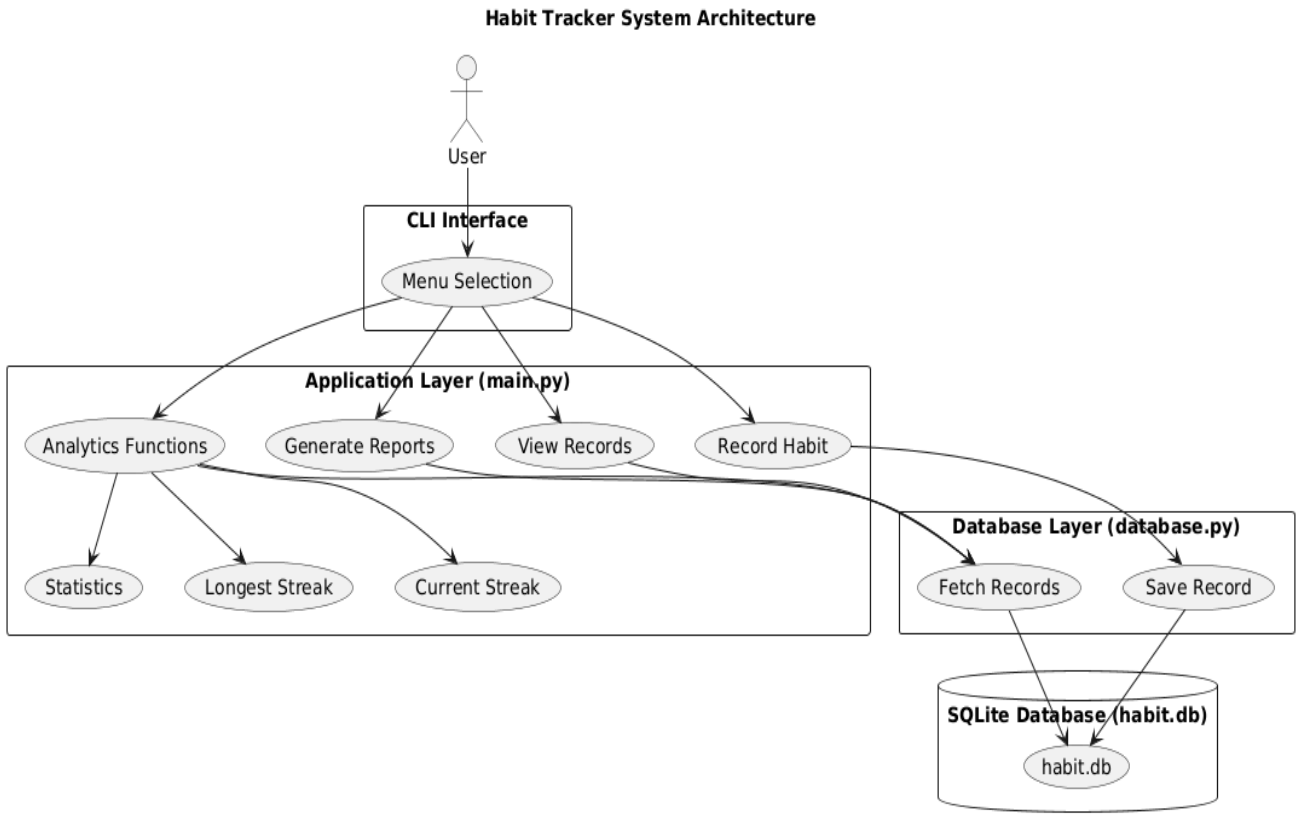
Habits are defined using a structured dictionary:

```
habits = {
    1: {"name": "Water Intake", "frequency": "daily", "unit": "gallons", "options": ["gallons
    2: {"name": "Exercise (Walking)", "frequency": "daily", "unit": "km", "options": ["km", '
    3: {"name": "Sleep", "frequency": "daily", "unit": "hours", "options": ["hours", "minutes
    4: {"name": "Reading", "frequency": "daily", "unit": "hours", "options": ["hours", "minut
    5: {"name": "Meditation", "frequency": "daily", "unit": "minutes", "options": ["minutes",
}
```

Event Log Structure

Each habit record includes:

- habit_id
- value
- unit
- date



STORAGE APPROACH AND DATA FLOW

Storage Approach

The application uses **SQLite database** instead of JSON.

- **Why SQLite?**
- persistent storage
- efficient querying
- scalable for future features

Data Flow

```
User Input
   ↓
main.py (logic)
   ↓
database.py (save / retrieve)
   ↓
SQLite (habit.db)
```

```
Select option: 6

Habit Records
Water Intake 2.0 gallons 2026-03-16
Water Intake 4.0 liters 2026-03-16
Water Intake 3.0 liters 2026-03-16
Water Intake 3.0 gallons 2026-03-16
Exercise (Walking) 7.0 km 2026-03-16
Exercise (Walking) 2.0 km 2026-03-16
Exercise (Walking) 4.0 miles 2026-03-16
Sleep 6.0 hours 2026-03-16
Sleep 60.0 minutes 2026-03-16
Sleep 4.0 hours 2026-03-16
Sleep 45.0 minutes 2026-03-16
Reading 56.0 minutes 2026-03-16
Reading 2.0 hours 2026-03-16
Meditation 35.0 minutes 2026-03-16
Meditation 1.0 hours 2026-03-16
Meditation 1.5 hours 2026-03-16
Water Intake 2.0 gallons 2026-03-17
Water Intake 4.0 liters 2026-03-17
Exercise (Walking) 7.0 km 2026-03-17
Exercise (Walking) 4.0 miles 2026-03-17
```

CLI WORKFLOW

User Interaction Flow

- Select menu option
- Choose habit
- Enter value + unit
- Data saved to database
- View analytics

User Interaction Flow

```
Create Habit Record
        ↓
Select Habit ID
        ↓
Enter Value + Unit
        ↓
Save to Database
        ↓
View Reports / Analytics
```

```
================================
        HABIT TRACKER
================================
1 View Habits
2 Record Habit
3 Longest Streak
4 Current Streak
5 Streak Statistics
6 View Records
7 Daily Report
8 Weekly Report
9 Monthly Report
10 Habit Leaderboard
11 Exit
Select option:
```

ANALYTICS FUNCTIONS

Implemented Functions: longest_streak(), current_streak(), streak_statistics(), daily_report(), weekly_report(), monthly_report(), habit_leaderboard()

Design Approach

- functions operate on database data
- no global state dependency
- reusable and modular

Status

All analytics functions are:

- ✓ implemented
  ✓ tested
  ✓ integrated with database

```
Habit ID: 1
Current streak: 1


Habit ID: 1
Longest streak: 2


Habit ID: 1
Max streak: 2
Min streak: 1
Average streak: 1.2
```

PREDEFINED HABITS AND TEST DATA

Predefined Habits

- Water Intake
- Exercise
- Sleep
- Reading
- Meditation

Dummy Data (4 Weeks)

- Dummy data was generated to simulate real usage:
- 30+ days of entries
- varying completion rates
- used for testing reports and streaks

```
Select option: 1

ID   Name                      Frequency
---------------------------------------------
1    Water Intake              daily
2    Exercise (Walking)        daily
3    Sleep                     daily
4    Reading                   daily
5    Meditation                daily


 Weekly Report
 ------------
 Water Intake      28% (2/7)
 Exercise (Walking)  28% (2/7)
 Sleep             28% (2/7)
 Reading           28% (2/7)
 Meditation        28% (2/7)
```

TESTING STRATEGY AND RESULTS

Testing Approach

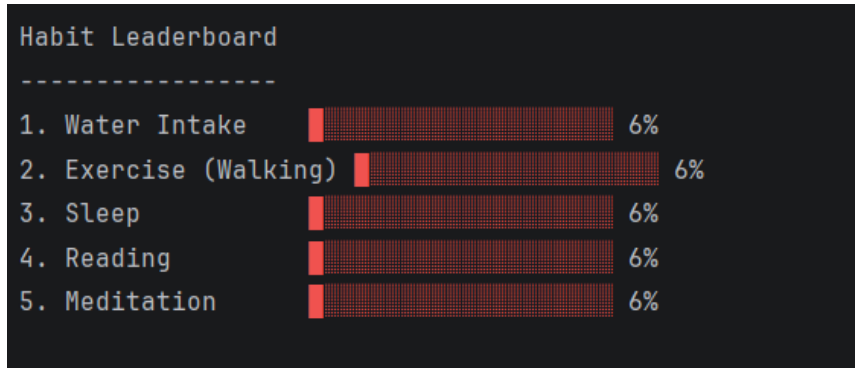
- manual testing via CLI
- automated testing using pytest
- dummy dataset validation

Test Cases

- batch detection of records
- performance tests
- database read/write tests

Results

- database persistence works
- streak calculations accurate
- reports generate correctly



PROGRESS VS ORIGINAL PLAN

Planned (Phase 1)

- simple CLI tracker
- basic habit recording
- no persistence

Achieved (Phase 2)

- ✓ SQLite database integration
  ✓ modular architecture
  ✓ full analytics system
  ✓ reports (daily, weekly, monthly)
  ✓ leaderboard + progress bars

- **Improvement Summary**
- The project evolved from a basic tracker → full analytics system

REFLECTION AND FUTURE IMPROVEMENTS

Key Learnings - This project improved skills in:

- Python programming
- database integration
- modular application design
- data analysis

Future Improvements - Possible extensions include:

- graphical user interface (GUI)
- user-defined habits
- data visualisation with charts
- mobile or web interface
- notification reminders

Conclusion

- The Habit Tracker demonstrates how Python can be used to create a functional productivity tool with analytics and persistent data storage.